

Technical Requirements Document (TRD)

Project Name

WorkForce360

Version

Version 1.0 (MVP)

Document Purpose

This Technical Requirements Document (TRD) defines the technical architecture, technology stack, database strategy, authentication mechanisms, APIs, deployment strategy, security requirements, performance requirements, and third-party integrations required to build WorkForce360.

The document serves as the technical blueprint for implementation and development.

1. Technical Overview

System Description

WorkForce360 is a web-based workforce, payroll, invoice, contract, and expense management platform designed specifically for staffing agencies.

The platform digitizes employee management, attendance tracking, payroll processing, document storage, contract management, invoice generation, and financial reporting.

The system follows a modern client-server architecture using Next.js for the frontend, Express.js for the backend, PostgreSQL as the database, and Cloudinary for document storage.

2. Technology Stack

Frontend Stack

Framework

Next.js

Purpose:

- Application UI
- Routing
- Page rendering
- State management integration

Reason:

- Production-ready architecture
 - Scalable folder structure
 - Excellent developer experience
-

Styling

Tailwind CSS

Purpose:

- UI styling
- Responsive layouts

Reason:

- Rapid development
 - Consistent design system
-

UI Components

Shadcn/UI

Purpose:

- Reusable UI components
- Dashboard components

Reason:

- Modern UI
 - Accessibility support
 - Faster development
-

HTTP Client

Axios

Purpose:

- API communication

Reason:

- Better error handling
 - Request interceptors
-

Analytics Visualization

Recharts

Purpose:

- Attendance graphs
- Payroll analytics
- Revenue and expense charts

Reason:

- React-friendly
 - Dashboard focused
-

Backend Stack

Runtime

Node.js

Purpose:

- Server execution environment

Reason:

- High productivity
 - Large ecosystem
-

Framework

Express.js

Purpose:

- REST API development

Reason:

- Lightweight
 - Easy maintenance
-

ORM

Prisma ORM

Purpose:

- Database communication
- Query abstraction
- Schema migrations

Reason:

- Type-safe database operations
 - Excellent PostgreSQL support
-

Database

Primary Database

PostgreSQL

Purpose:

- Store all application data

Reason:

- Strong relational support
 - ACID compliance
 - Suitable for payroll and financial systems
-

Authentication

Authentication Method

JWT (JSON Web Tokens)

Purpose:

- User authentication
 - Session management
-

Password Security

bcrypt

Purpose:

- Password hashing

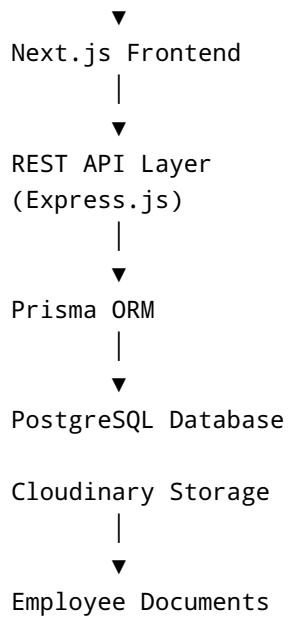
Reason:

- Industry standard
 - Secure password storage
-

3. System Architecture

High-Level Architecture

```
Admin / Worker
|
```



Architecture Pattern

Monolithic Modular Architecture

Reason:

- Single admin system
- Small user base
- Faster development
- Easier deployment
- Lower maintenance

4. Frontend Architecture

Main Modules

Authentication Module

Pages:

- Login
- Change Password

Dashboard Module

Admin Dashboard

Worker Dashboard

Worker Management Module

Pages:

- Employee List
 - Employee Profile
 - Employee Onboarding
 - Employee Documents
-

Attendance Module

Pages:

- Daily Attendance
 - Attendance History
 - Attendance Reports
-

Payroll Module

Pages:

- Payroll Generation
 - Payroll History
 - Salary Revisions
-

Payslip Module

Pages:

- Payslip Viewer
 - Payslip Downloads
-

Client Module

Pages:

- Client List
 - Client Details
-

Contract Module

Pages:

- Contracts
 - Contract History
-

Invoice Module

Pages:

- Generate Invoice
 - Invoice History
-

Expense Module

Pages:

- Add Expense
 - Expense Reports
-

Analytics Module

Pages:

- Revenue Reports
 - Profit Reports
 - Payroll Reports
-

5. Backend Architecture

API Layers

Authentication Layer

Responsibilities:

- Login
 - Password validation
 - JWT generation
-

Employee Layer

Responsibilities:

- Employee onboarding
 - Employee updates
 - Employee retrieval
-

Attendance Layer

Responsibilities:

- Attendance creation
 - Attendance summaries
 - Overtime tracking
-

Payroll Engine

Responsibilities:

- Salary calculations
 - PF calculations
 - ESIC calculations
 - PT calculations
 - Employer contributions
-

Invoice Engine

Responsibilities:

- Invoice calculations
 - GST calculations
 - Service charge calculations
-

Expense Engine

Responsibilities:

- Expense recording
 - Expense reporting
-

Report Engine

Responsibilities:

- Analytics generation
 - PDF reports
 - Excel exports
-

6. Database Architecture

Core Entities

Users

Stores:

- Admin Accounts
 - Worker Accounts
-

Employees

Stores:

- Personal Details
- Employment Details
- Salary Information

Family Details

Stores:

- Parent Details
- Spouse Details
- Children Details

Education Records

Stores:

- Qualification History

Salary Revisions

Stores:

- Salary History
- Effective Dates

Attendance

Stores:

- Daily Attendance
- Overtime Hours

Payroll

Stores:

- Monthly Payroll Snapshots

Payroll Items

Stores:

- Salary Breakdown

Documents

Stores:

- Cloudinary References

Clients

Stores:

- Client Information

Contracts

Stores:

- Service Charges
- GST Settings
- Contract Terms

Invoices

Stores:

- Monthly Invoice Records

Expenses

Stores:

- Automatic Expenses
 - Manual Expenses
-

7. Authentication Design

User Types

Admin

Permissions:

- Full Access
-

Worker

Permissions:

- Read-only access
-

Login Flow

1. User enters credentials
 2. Credentials validated
 3. Password checked using bcrypt
 4. JWT generated
 5. Token returned
 6. Protected routes unlocked
-

8. APIs Required

Authentication APIs

```
POST /api/auth/login
POST /api/auth/change-password
POST /api/auth/reset-password
```

Employee APIs

```
GET /api/employees
GET /api/employees/:id
```

```
POST /api/employees
PUT /api/employees/:id
```

Attendance APIs

```
POST /api/attendance
GET /api/attendance/monthly
GET /api/attendance/employee/:id
```

Payroll APIs

```
POST /api/payroll/generate
GET /api/payroll/history
GET /api/payroll/:id
```

Payslip APIs

```
GET /api/payslips/:id
GET /api/payslips/download/:id
```

Client APIs

```
GET /api/clients
POST /api/clients
PUT /api/clients/:id
```

Contract APIs

```
GET /api/contracts
POST /api/contracts
PUT /api/contracts/:id
```

Invoice APIs

```
POST /api/invoices/generate  
GET /api/invoices  
GET /api/invoices/:id
```

Expense APIs

```
POST /api/expenses  
GET /api/expenses
```

Analytics APIs

```
GET /api/reports/payroll  
GET /api/reports/attendance  
GET /api/reports/profit  
GET /api/reports/revenue
```

9. File Storage Architecture

Storage Provider

Cloudinary

Documents Stored

- Employee Photo
 - Aadhaar
 - PAN
 - Passbook
 - Joining Form
 - PF Documents
 - ESIC Documents
-

Database Storage

Only URLs stored in PostgreSQL.

Actual files remain in Cloudinary.

10. PDF Generation

Tool

Puppeteer

Generated PDFs

- Payslips
 - Invoices
 - Reports
-

11. Excel Export

Tool

ExcelJS

Exportable Reports

- Attendance
 - Payroll
 - Expenses
 - Revenue
 - Profit
 - Invoices
-

12. Deployment Architecture

Frontend

Platform: Vercel

Backend

Platform: Render

Database

Platform: Supabase PostgreSQL

File Storage

Platform: Cloudinary

13. Security Requirements

Authentication Security

- JWT Authentication
 - Role-Based Access Control
-

Password Security

- bcrypt hashing
 - Password update functionality
-

API Security

- Input validation
 - Request sanitization
 - Rate limiting
-

File Security

- Allowed file types only
 - File size restrictions
 - Secure Cloudinary uploads
-

Data Security

- Environment variables
 - Secure database credentials
 - HTTPS-only deployment
-

14. Performance Requirements

Application Performance

- Dashboard load time < 3 seconds
 - API response time < 500 ms
 - Payroll generation < 30 seconds
 - Invoice generation < 15 seconds
-

Scalability Requirements

Support:

- 500+ employees
 - Multiple clients
 - Multiple contracts
 - Historical payroll records
-

15. Third-Party Integrations

Cloudinary

Purpose: Document storage

Supabase PostgreSQL

Purpose: Managed PostgreSQL database

Puppeteer

Purpose: PDF generation

ExcelJS

Purpose: Excel export generation

16. Technical Decisions and Justifications

Decision	Selected Technology	Reason
Frontend Framework	Next.js	Scalable and production-ready
UI Styling	Tailwind CSS	Rapid UI development
UI Components	Shadcn/UI	Professional dashboard UI
Backend Framework	Express.js	Lightweight and flexible
Database	PostgreSQL	Strong relational support
ORM	Prisma	Type-safe database operations
Authentication	JWT + bcrypt	Secure and industry standard
File Storage	Cloudinary	Easy and scalable storage
PDF Engine	Puppeteer	High-quality PDF generation
Excel Engine	ExcelJS	Advanced spreadsheet exports
Frontend Hosting	Vercel	Optimized for Next.js
Backend Hosting	Render	Easy deployment
Database Hosting	Supabase	Managed PostgreSQL service

Future Technical Enhancements

- Mobile Application
- Email Notifications
- SMS Notifications
- QR Attendance
- Biometric Integration
- AI Analytics
- Multi-Admin Support
- Client Portal
- Audit Logging Dashboard
- Cloud Backup Automation